Optimization for Deep Learning (OPT4DL)

Abolfazl Hashemi, Purdue ECE

Deep Learning Architectures 3: RNNs

Learning with Sequence Data



- Recall, we discussed the connection between ERM and MLE

$$\min_w f(w) := \frac{1}{n}\sum_{i=1}^{n} \ell(w,(x_i,y_i)) \quad \Leftrightarrow \quad \max_w \prod_{i=1}^{n} p(y_i|x_i;w) \tag{1}$$

  in the context of classification and regression

- Many applications we work with sequences $x_1, x_2, \ldots$ where $x_i \in \mathbb{R}^p$ are observations at time t (e.g., working with financial data, language modeling, DNA data, etc.)
- Natural to assume a causal model where each observation depends on the previous ones $x_t = A(x_1, \ldots, x_{t-1}; w^*)$. An approach referred to as auto-regressive (AR) models
- Natural to follow a similar likelihood maximization approach to find w^*. For instance,

$$x_{t+1} = w(1)x_1 + w(2)x_2, \ldots, +w(t)x_t \tag{2}$$

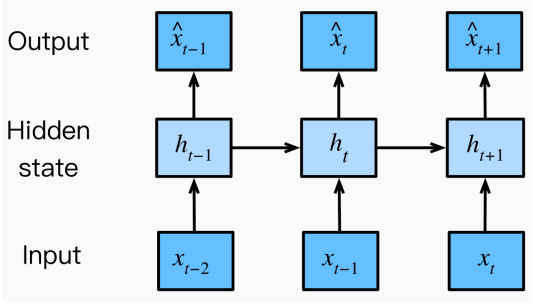
- Issue: too many features, i.e. $\mathcal{O}(t)$ when t grows

Latent Modeling

- A solution is to limit the dependency of x_t to only $x_{t-\tau}, \ldots, x_{t-1}$. More generally, we aim to find a latent variable that effectively summarizes the dependencies of the observations

$$x_t = A(h_{t-1}; w^*), \qquad h_t = g(h_{t-1}, x_t; \omega^*) \tag{3}$$

- We can represent these functions by NNs and learn their parameters by fitting the data

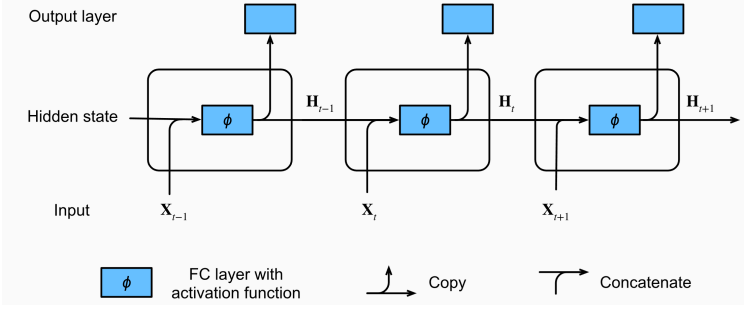


Simple RNN

- For instance, let us resort to MLPs $h_i = \sigma(W_i h_{i-1} + b_i), \quad i = 1, \ldots, L, \quad h_0 = x$
- For latent AR modeling

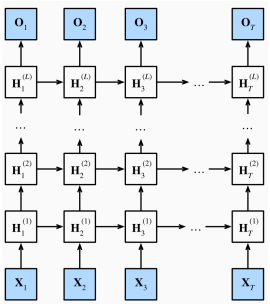
$$h_t = g(h_{t-1}, x_t; \omega^*) \quad \Leftrightarrow \quad h_t = \sigma(W h_{t-1} + b + \tilde{W} x_t) \tag{4}$$

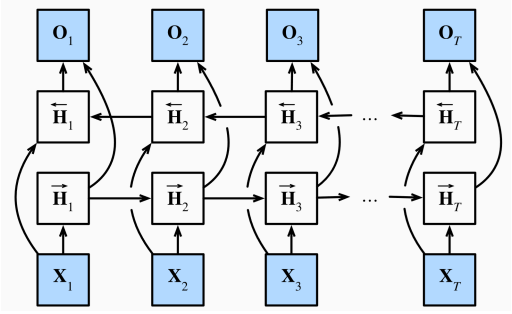
called an RNN with one hidden layer where $\omega^* = (W, \tilde{W}, b)$. Notably, the parametrization cost of an RNN does not grow as the number of time steps increases.



RNN++

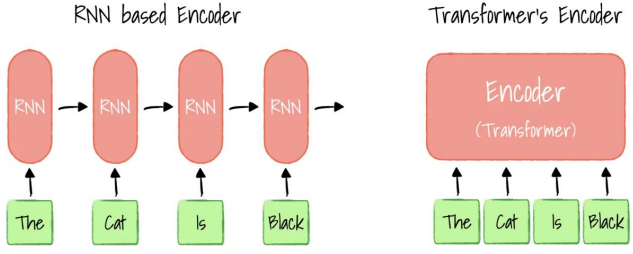
- Certain improvements to RNN, e.g. LSTM, GRU, to deal with the information/signal loss when working with long sequences and long-term dependencies. Solutions similar in nature to skip/residual connections
- We can also stack RNNs and form multiple hidden layers
- We can process the sequences from both directions to learn the dependencies more effectively, e.g. bidirectional LSTM





From RNNs to Transformers

- RNNs like CNNs and MLP need to work with data of a predefined size
- For RNNs, the sequence length should then be fixed. Thus, could limit their applicability
- RNNs further suffer from the need for sequential computation, not able to leverage advances parallel computing
- Transformers leverage the attention mechanism and positional encoding to alleviate these issues (at the cost of higher parallel computational cost)



Attention Mechanism

- Idea based on databases, i.e. collections of keys K and values v
- We can make a query q regardless of the database size
- Instead of returning one of the values in the database, attention typically returns

$$\text{Attention}(q, D) = \sum_{i=1}^{m} \alpha(q, k_i) v_i, \quad \alpha(q, k_i) = \frac{exp(\langle q, k_i \rangle / \sqrt{d})}{\sum_{j=1}^{m} exp(\langle q, k_j \rangle / \sqrt{d})} \tag{5}$$

  i.e., using softmax to ensure weights are nonnegative and sum to one (convex combination).
- In practice, q, v, k are matrices and include learnable weight matrices.

